

一、单选题

1、下面有关 C++的一些叙述中，错误的有（ ）。

A)如果通过基类指针调用虚函数，C++就会根据指针所指向的对象类型(而不是指针本身的类型)，在运行时确定实际执行的函数。通过基类引用调用虚函数也会达到相同的效果。

B)只有类的成员函数可以声明为虚函数，而全局(普通)函数、类的构造函数和析构函数、以及静态成员函数不能声明为虚函数。

C)用基类指针又不想进行动态绑定时可以在虚函数前加类名::限定。

D)一旦将函数说明为虚函数，不管它通过多少层派生，它都是虚函数。派生类重新定义虚函数时，不需要重复使用关键字 `virtual`。

2、下述程序的输出结果为：（ ）。

```
#include <iostream>
int & min(int &x,int &y) { return (x<y?x:y);}
int main() {
    int a=10,b=20,c=30;
    min(a,b)=5;
    min(a,c)=6;
    min(b,c)=7;
    std::cout<<a<<"\t"<<b<<"\t"<<c<<std::endl;
}
```

A. 出错

B. 6 7 30

C. 5 6 7

D. 10 20 30

3、下面有关 C++的一些叙述中，错误的有（ ）。

A)当类 `MyClass` 的构造函数只有一个参数时，下述的 3 种方法都可以进行对象的初始化：`MyClass t1(8);MyClass t2 = MyClass(8);MyClass t3 = 8;`

B)`MyClass t;`该语句在下述 3 种情况下正确：类 `MyClass` 没有构造函数(此时使用默认构造函数)；类 `MyClass` 定义有无参的构造函数；类 `MyClass` 的某一个构造函数允许所有参数全部为默认值。

C)`Myclass obj;`和 `Myclass obj();`等价，他们都是调用类 `MyClass` 的无参构造函数创建一个对象。

D)有定义：`Myclass obj,*pObj=&obj;`则可以用：`(*pObj).func(10);`执行类 `Myclass` 的成员函数 `func`。也可以用：`pObj->data` 访问类 `Myclass` 的数据成员 `data`。

4、类成员指针变量不是类的成员，它只是程序中的一个指针变量，所指向的成员必须具有 `public` 访问权限。下述程序只有（ ）所在行会导致编译错。

```
#include <iostream>
struct MyClass { int a,b; };
int main() {
    MyClass obj;
```

```

obj.a=obj.b=88; // 【A】
int MyClass::*p; // 【B】
p=MyClass::&b; // 【C】
std::cout<<obj.a<<','<<obj.*p; // 【D】
}

```

5、下述程序只有（ ）所在行会调用类的拷贝构造函数。

```

#include <iostream>
class MyClass {
    int m;
public:
    MyClass(int x=0) { m=x; }
    const MyClass &f1(const MyClass &x) const
    { return x; }
    MyClass &f2(MyClass *p) const { return *p; }
};
int main() {
    MyClass y;
    MyClass x=y; // 【A】
    x=y; // 【B】
    x.f1(y); // 【C】
    x.f2(&y); // 【D】
}

```

6、下述程序的输出结果是：（ ）。

```

#include <iostream>
struct Ca {
    ~Ca() {std::cout<<"~Ca ";} };
struct Cb: virtual public Ca { ~Cb() {std::cout<<"~Cb ";} };
struct Cc: virtual public Ca { ~Cc() {std::cout<<"~Cc ";} };
struct Cd: public Ca { ~Cd() {std::cout<<"~Cd ";} };
struct Ce {
    ~Ce() {std::cout<<"~Ce ";} };
struct Cf: public Cd,public Cc,public Cb {
    Ce e;
    ~Cf() {std::cout<<"~Cf ";}
};
int main() { Cf obj; }

```

A、~Cf ~Ce ~Cb ~Cc ~Cd ~Ca ~Ca B、~Cf ~Ce ~Cb ~Cc ~Ca ~Cd ~Ca
C、~Ca ~Cd ~Ca ~Cc ~Cb ~Ce ~Cf D、~Cf ~Ce ~Cb ~Ca ~Cc ~Ca ~Cd ~Ca

7、下述程序只有（ ）所在行会导致编译错。

```

class MyClass {
    int a,b;
public: MyClass(int a,int b=0) {
    MyClass::a=a; // 【A】
}
}

```

```

        this->b=b; // 【B】
    }
};
int main() {
    MyClass *p;
    p=new MyClass(6); // 【C】
    p=new MyClass[6]; // 【D】
}

```

8、下列关于 `const` 的程序中，只有程序行（ ）不会导致编译错。

```

#include <iostream>
int main()
{
    const int ci=1;
    int i=2,k=3;
    int * const pc0;    // 【A】
    int * const pc1=&i; // 【B】
    int * const pc2=&ci; // 【C】
    pc1=&k;             // 【D】
}

```

9、下述程序只有（ ）所在行不会导致编译错，也不会导致运行时错误。

```

#include <iostream>
class B {
protected:
    int b1,b2;
public:
    B(int x=0,int y=0) {b1=x;b2=y;}
    void show() const {std::cout<<b1<<' \n'<<b2<<std::endl;}
    void show(int t) const {
        if (t==1) std::cout<<' B'<<b1<<std::endl;
        else std::cout<<' B'<<b2<<std::endl;
    }
};
class D : public B {
    int d;
public:
    D(int x=0,int y=0,int z=0):B(x,y) {d=z;}
    void show() {
        B::show();
        std::cout<<' D'<<d+b2<<std::endl; // 【A】
    }
};
int main() {

```

```

    B * bp;
    const D d(1, 2, 3);
    d.show(); // 【B】
    d.show(1); // 【C】
    bp=&d; // 【D】
    return 0;
}

```

10、下述程序只有()所在行会导致编译错。

```

#include <iostream>
class C {
    static int i; // 【A】
public:
    C(int x=0) { i=x; }
    static int get() const { return i; } // 【B】
};
int C::i=999; // 【C】
int main() {
    C x(1);
    std::cout<<x.get()<<', '<<C::get()<<std::endl; // 【D】
    return 0;
}

```

二、填空题

1、下述程序的输出结果是：_____。

```

#include <iostream>
using namespace std;
int main() {
    const long x=12345;
    cout.fill('#');
    cout.flags(cout.flags()|ios::right);
    cout.setf(ios::showpos);
    cout.width(7);
    cout<<x<<', '<<x<<endl;
}

```

2、下述程序的输出结果是：_____。

```

#include <iostream>
using namespace std;
class Base {
    int bi;
public:
    Base() {bi=0;}
    ~Base() {cout<<"B["<<bi<<"};}
}

```

```

};
class Base1 {
    int bi1;
public:
    Base1(int x)    {bi1=x;}
    ~Base1()        {cout<<"B1["<<bi1<<"]";}
};
class Base2 {
    int bi2;
public:
    Base2(int x=8) {bi2=x;}
    ~Base2()       {cout<<"B2["<<bi2<<"]";}
};
class Myclass {
    int mi;
public:
    Myclass(int x) {mi=x;}
    ~Myclass()     {cout<<"Mc["<<mi<<"]";}
};
class Derived:public Base2,Base1,protected Base {
    Myclass obj;
    int di;
public:
    Derived(int a,int b,int c,int d)
        :di(a),obj(b),Base1(c),Base2(d) {}
    ~Derived()    {cout<<"D["<<di<<"]";}
};
int main()        {Derived dObj(1,2,3,4);}

```

3、下述程序的输出结果是：_____。

```

#include <iostream>
#include <typeinfo>
using namespace std;
struct Base {
    virtual void show() const {cout<<"B"<<' ';}
};
struct Derived:Base {
    virtual void show() const {cout<<"D"<<' ';}
};
void showAll(const Base &br) { //普通(全局)函数
    br.show();
}
int main() {
    Base    bObj,*bp=&bObj;
    Derived dObj,*dp=&dObj;
    bp->show();    dp->show();
}

```

```

    bp=&dObj;    bp->show();
    showAll(bObj);showAll(dObj);
    cout<<(typeid(*bp)==typeid(*dp))<<endl;
}

```

4、在形如: `Complex(const double &r);` 的单参数构造函数前加上_____关键字可以禁止隐式类型转换。但使用显式类型转换仍然可以进行类型转换。

5、_____是一个在基类中说明的虚函数，但未给出具体的实现，要求在其派生类中实现。包含这种函数的基类(称为抽象类)仅用于继承，作为一个接口(interface, 界面)，具体功能在其派生类中实现。抽象类只能作为基类用来派生新类，而不能用来创建对象。

6、在题目的横线上填上适当的语句完成下述程序，该程序的输出结果是：_____。

```

#include <iostream>
class Myclass {
    static int i;
public: Myclass() {
        i++;
        std::cout<<"i1="<<i<<' ';
    }
    ~Myclass() {
        i--;
        std::cout<<"i2="<<i<<' ';
    }
    static void show() { std::cout<<"i3="<<i<<' '; }
};
//(学生填写)定义静态变量 i，并初始化其值为 0；

```

```

int main() {
    Myclass x,y;
    Myclass::show();
    x.show();
    y.show();
}

```

7、x 和 y 是 2 个已经定义的对象，下述语句判断这 2 个对象是否类型一致：

if(_____) cout<<"x 和 y 类型一样。\\n";

8、下述程序的输出结果是：_____。

```

#include <iostream>
using namespace std;
struct Base { virtual void show() const { cout<<"B.."; } };
struct Derived1:Base { void show() const { cout<<"D1."; } };
struct Derived2:Base { virtual void show() const; };
void Derived2::show() const { cout<<"D2."; }
struct Derived3:Base { virtual void show(int x) const { cout<<"D3:"<<x<<". "; } };
struct Grandson:Derived2 { };
void showAll(const Base &br) { br.show(); }

```

```

int main() {
    Base bObj,*bp=&bObj;
    Derived1 dObj1;
    Derived2 dObj2;
    Derived3 dObj3;
    Grandson gObj;
    bp=&dObj2;
    bp->show();
    showAll(bObj);
    showAll(dObj1);
    showAll(dObj2);
    showAll(dObj3);
    showAll(gObj);
}

```

9、下述程序的输出结果是：_____。

```

#include <iostream>
#include <iomanip>
int main(){
    std::cout.precision(4);
    std::cout.fill("#");
    std::cout << std::setw(7) << 123 << "," << 123.2 << "\n";
}

```

10、可将自定义的类型转换为其他数据类型进行混合运算，在下述横线处填上正确的语句实现该功能。下述程序输出：3。

```

#include <iostream>
struct MyClass {
    int a,b;
    MyClass(int x,int y):a(x),b(y) { };
    _____{ return a+b; }
};
int main() {
    MyClass obj(1,2);
    std::cout<<(int)obj;
}

```

三、是非题

- 1、()C++ 允许声明引用的数组，也允许声明指向数组的引用。如：int x=0,y=1,z=2;int &s[3]={x,y,z};int a[]={1,2,3,4};int (&r)[4]=a;
- 2、() 在 C++中，可以为函数参数指定默认值，在函数声明和定义都必须指定默认参数。
- 3、() this 指针是所有（类）成员函数的隐含参数。因此在成员函数内部可以直接使用 this 指针引用调用对象。类中成员函数有了隐含指针变量 this 后，就可以保证用不同的对象调用成员函数是对不同对象的操作。

- 4、() 有声明 `bool b`; 则 `b=(y%4==0&& y%100!=0|| y%400==0); b=1; b=false`; 等语句均是正确的。
- 5、() 下述的 2 个语句含义是完全一致的: `Myclass obj; Myclass obj()`;
- 6、() 构造函数不能声明为虚函数, 而析构函数却可以声明为虚函数。例如: `virtual ~Base(){ delete[] pb; }`。
- 7、() 当类 `MyClass` 有单参数的构造函数时, 则下述的 3 种方法都可以进行对象的初始化: `MyClass t1(8); MyClass t2 = MyClass(8); MyClass t3 = 8;`
- 8、() 如果重载过运算符 `==`, 没有重载过运算符 `!=`, 则可以用 `if (a==b)...; if (a!=b)...; if (!(a==b))...` 来判断两个对象 `a` 和 `b` 是否相等, 并且后面的两个判别条件是等价的。
- 9、() 类型转换函数也可以被重载, 如, 类里面可以同时有: `operator long() { return ld; }` 和 `operator double() { return ld*1.0; }` 这样的两个类型转换函数存在。
- 10、() 对象按引用/指针调用或返回对象引用/指针时 (无论前面是否加 `const`), 由于不创建对象的副本, 所以并不调用类的拷贝构造函数, 当然也就不会调用类的析构函数, 所以执行速度较快。

四、程序填空题

- 1、下述程序输出: 8, 请填空。

```
#include <iostream>
class MyClass {
public:
    MyClass(int x=0) {MyClass::x=x;}
    _____{
        std::cout<<x<<std::endl;
    }
private:
    int x;
};
int main() {
    const MyClass x(8), *p=&x;
    (*p).show();
    return 0;
}
```

- 2、下述程序为定义圆类的片段, 请在下划线处填写构造函数的程序(行)。

```
#include <iostream>
class Circle {
public:
    //构造函数
    _____
    //.....
private:
    const double PI; //类的常数据成员
    double radius; //圆的半径
};
```



```
int main() {
    Circle r;          //声明圆类对象 r，默认构造(圆的半径=1)
    //.....
}
```

3、下面的程序输出：3，请填空完成程序。

```
#include <iostream>
using namespace std;
class Myclass {
    static int cnt;
public:
    Myclass() {cnt++;}
    static void show(void) {cout<<cnt<<endl;}
};
```

_____;

```
int main() {
    Myclass s1,s2,s3;
    s2.show();
}
```

4、在 C++语言中，除了可以定义对象指针外，还可以定义类成员指针。类成员指针包括类数据成员指针和类函数成员指针。类成员指针变量不是类的成员，它只是程序中的一个指针变量。所指向的成员必须具有 **public** 访问权限。下述程序输出 888,888。请按照程序的注解完成下述的程序：

```
#include <iostream>
```

```
struct MyClass {
    int a,b;
};
```

```
int main() {
    //定义指向 int 型的类数据成员指针变量 p，同时使 p 指向类的数据成员 b
```

_____;

```
MyClass obj;    obj.b=888;
//利用类数据成员指针变量 p 输出 obj.b 的值
std::cout<<obj.b<<','<<obj.*p<<std::endl;
//.....
```

```
}
```

5、下述程序完成加行号并分屏显示 C 语言源程序，请填空。

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;
#define MAX_BUF 255
#define LINE_NUM 24
int main() {
    ifstream in;
```

```

//打开文件 f:\Temp\test.cpp，用于读出。
_____
if (_____)
    {printf("Error on open test.cpp file!"); return -1; }
char s[MAX_BUF];    int lineNo=0;
while (in.getline(s,MAX_BUF)) {
    //行号占 5 位，左对齐。
    _____;
    if (lineNo%LINE_NUM==0) getchar();
}
in.close();
}

```

6、d 盘 temp 文件夹下的文件 grade.dat 存放一批百分制成绩，下述程序输出这批成绩 的最高分，请填空完成该程序。

```

#include <iostream>
#include <fstream>
int main() {
    _____;
    if (!in) { std::cout<<"Cannot open file.\n"; return -1; }
    double max=-1,g=0;
    while (_____)
        if (g>max) max=g;
    std::cout<<"最高分:"<<max<<std::endl;
    in.close();
}

```

7、Assistant(助教类)从 Teacher(教师类)和 Student(学生类)公有派生，后两者均从 Person(人员类，虚基类)公有派生，请填空。

```

#include <iostream>
#include <cstring>
using namespace std;
class Person {
protected:
    char id[19];
public:
    Person(char *id) { strcpy(this->id,id); }
};
class Teacher:_____ {
protected:
    char teacherID[6];
public:
    Teacher(char *id,char *tID):Person(id) { strcpy(teacherID,tID); }
};

```

```

class Student;//定义省略，类包含有保护成员：char studentID[10];
class Assistant:public Teacher,public Student {
    public:
        Assistant(char *id,char *tID,char *sID) :
            _____{ }
};
int main() {
    Assistant obj("350102200001010101","87018","221801505");
}

```

8、假定“日期+整数”已经有定义，要求你填空完成下述程序对运算符++的重载。

```

class Date {
    public: //.....
        Date& operator++() { //重载前置单目运算符++
            _____
        }
};

```

9、填空完成下面的程序，程序应该正确输出：

```

Destructing Derived... Destructing Base...
#include <iostream>
class Base {
    char *pb;
public:
    Base(int size) {pb=new char[size];}
    _____ {
        std::cout<<"Destructing Base...\t";
        delete[] pb;
    }
};
class Derived:public Base {
    char *pd;
public:
    Derived(int size1,int size2)
        : _____{pd=new char[size2];}
    virtual ~Derived() {
        std::cout<<"Destructing Derived...\t";
        delete[] pd;
    }
};
int main() {
    Base *pb=new Derived(100,120);
    //.....
    delete pb;
}

```

